НАСЛЕДОВАНИЕ И ВИРТУАЛЬНЫЕ ФУНКЦИИ

1 Постановка задачи

1.1 Общая постановка задачи

В лабораторной работе требуется разработать библиотеку классов, представляющую иерархию государств, и демонстрационное приложение, использующее эту библиотеку. Базовые и производные классы должны содержать поля и свойства с проверкой допустимых значений, конструкторы без параметров, с параметрами и копирования, а также методы отображения, ввода, случайной инициализации и сравнения объектов.

Необходимо сформировать массив объектов различных классов и продемонстрировать полиморфный вызов методов. Отдельная часть работы посвящена реализации запросов к массиву объектов. Далее требуется включить в иерархию интерфейс `IInit`, выполнить сортировку и поиск с использованием `IComparable`, `IComparer`, метода `Array.Sort` и бинарного поиска. Также необходимо создать класс вне основной иерархии, реализующий `IInit`, и показать работу клонирования и поверхностного копирования.

1.2 Постановка задачи конкретного варианта

Вариант предусматривает следующую предметную область: государство, монархия, королевство и республика. В реализованном проекте базовым классом является `State`. Класс `Monarchy` наследуется от `State`, класс `Kingdom` наследуется от `Monarchy`, а класс `Republic` непосредственно наследуется от `State`.

Для объектов государства используются характеристики: название, столица, численность населения и площадь. Для монархии добавлены имя монарха и признак конституционной формы правления; для королевства — название династии; для республики — имя главы и срок его полномочий. Дополнительно реализован класс `City`, не входящий в иерархию государств и используемый для представления столицы.

Вторая часть варианта включает три запроса:

а) получение названия государства и названия его столицы для каждого объекта;

б) получение списка государств, являющихся монархиями;

в) получение имён глав республик, срок полномочий которых меньше заданного пользователем значения n.

В третьей части требуется сортировать массив государств по площади и численности населения, выполнять бинарный поиск по площади, демонстрировать работу `IInit` на объектах иерархии и класса `City`, а также сравнить поверхностное копирование с клонированием.

2 Разработка и реализация

2.1 Иерархия классов

На рисунке 2.1 представлена структура классов предметной области. Класс `State` является базовым. Класс `Monarchy` расширяет его характеристиками монархической формы правления, а `Kingdom` добавляет сведения о династии. Класс `Republic` содержит данные о главе республики и сроке его полномочий. Объект `City` связан с государством отношением «имеет столицу».

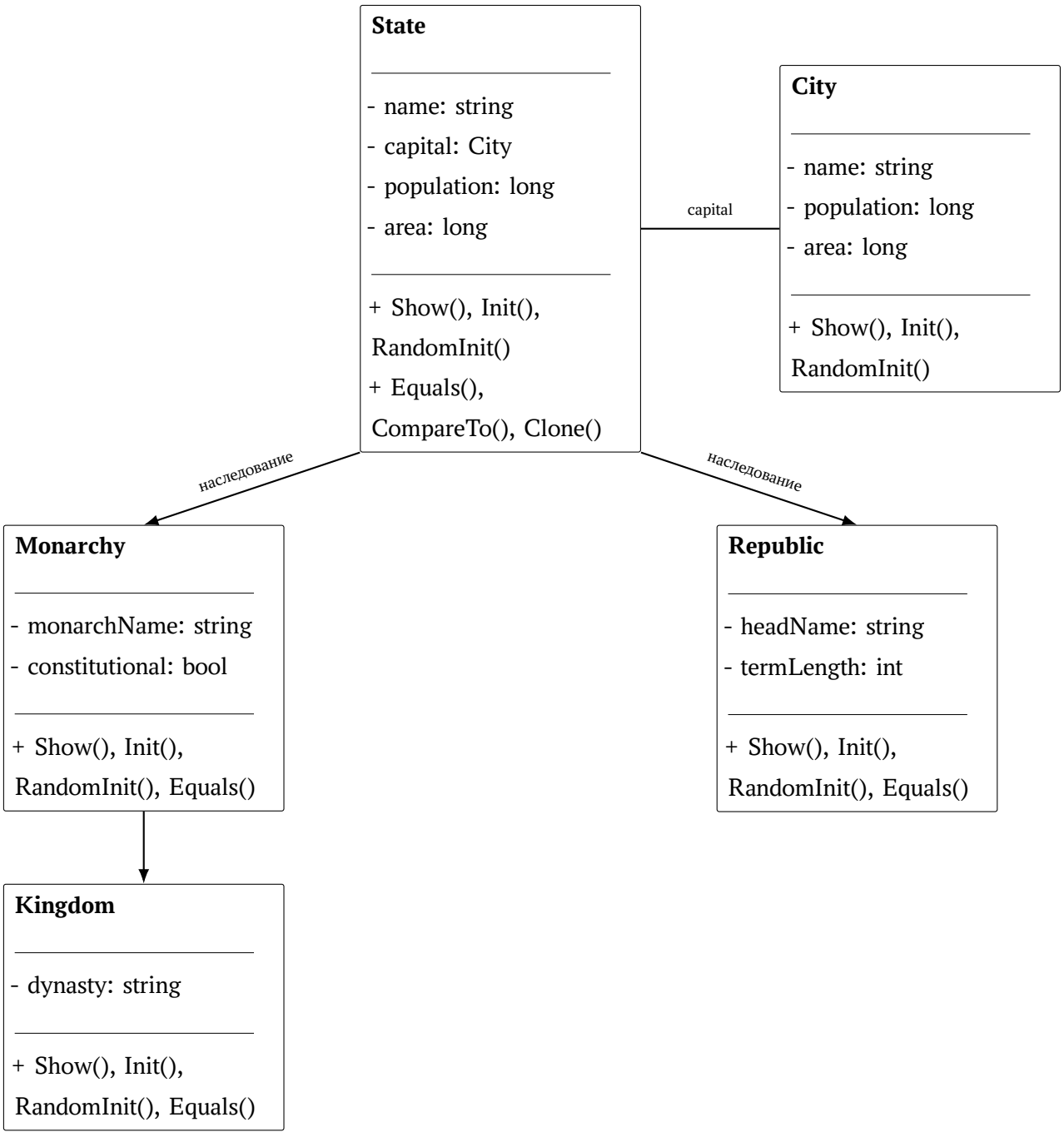


Рисунок 2.1 — Диаграмма классов предметной области

2.2 Диаграмма классов для запросов

Запросы вынесены в отдельный класс `Requests`. Он получает массив объектов `State` и формирует результаты трёх видов. Первый метод возвращает массив пар «государство — столица», второй отбирает объекты типа `Monarchy`, третий проверяет принадлежность объекта к `Republic` и сравнивает срок полномочий со значением n.

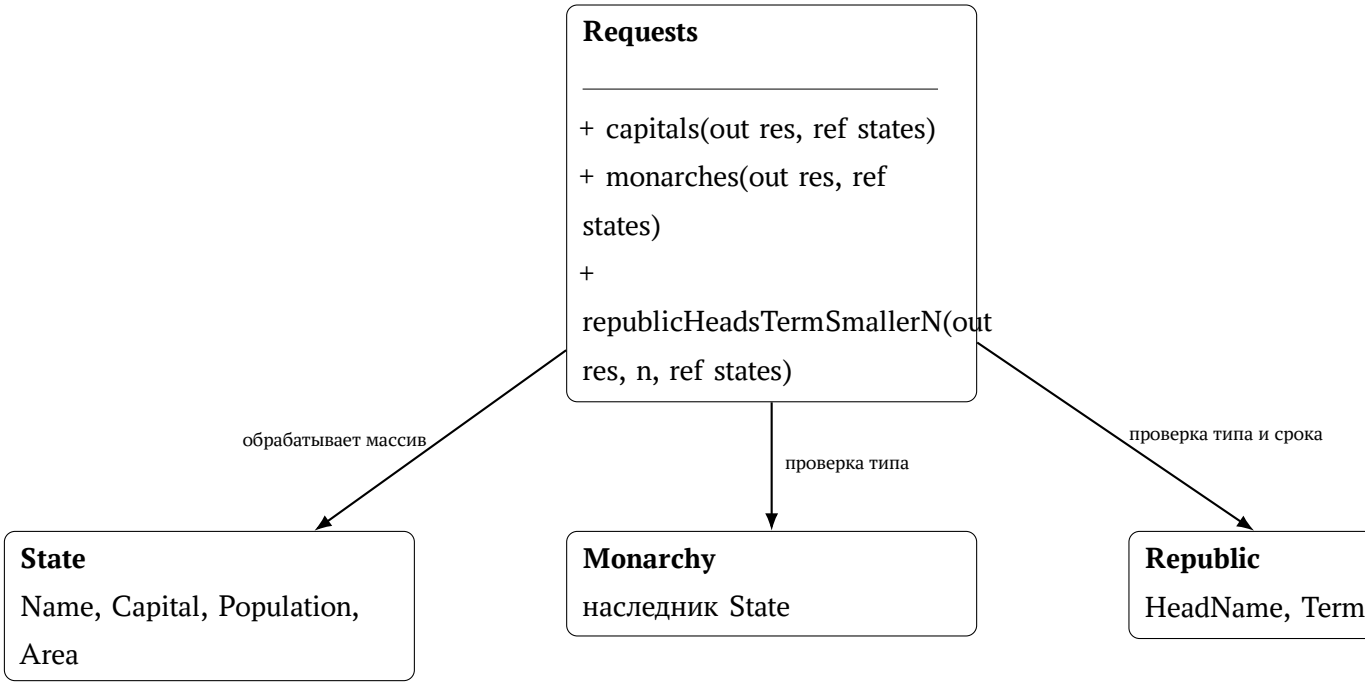


Рисунок 2.2 — Диаграмма взаимодействия класса запросов с иерархией

2.3 Диаграмма интерфейсов и средств сортировки

В третьей части базовый класс `State` реализует интерфейсы `IInit`, `IComparable<State>` и `ICloneable`. Класс `City` реализует `IInit`, хотя не входит в иерархию государств. Класс `PopulationComparer` реализует `IComparer<State>` и задаёт альтернативный критерий сортировки.

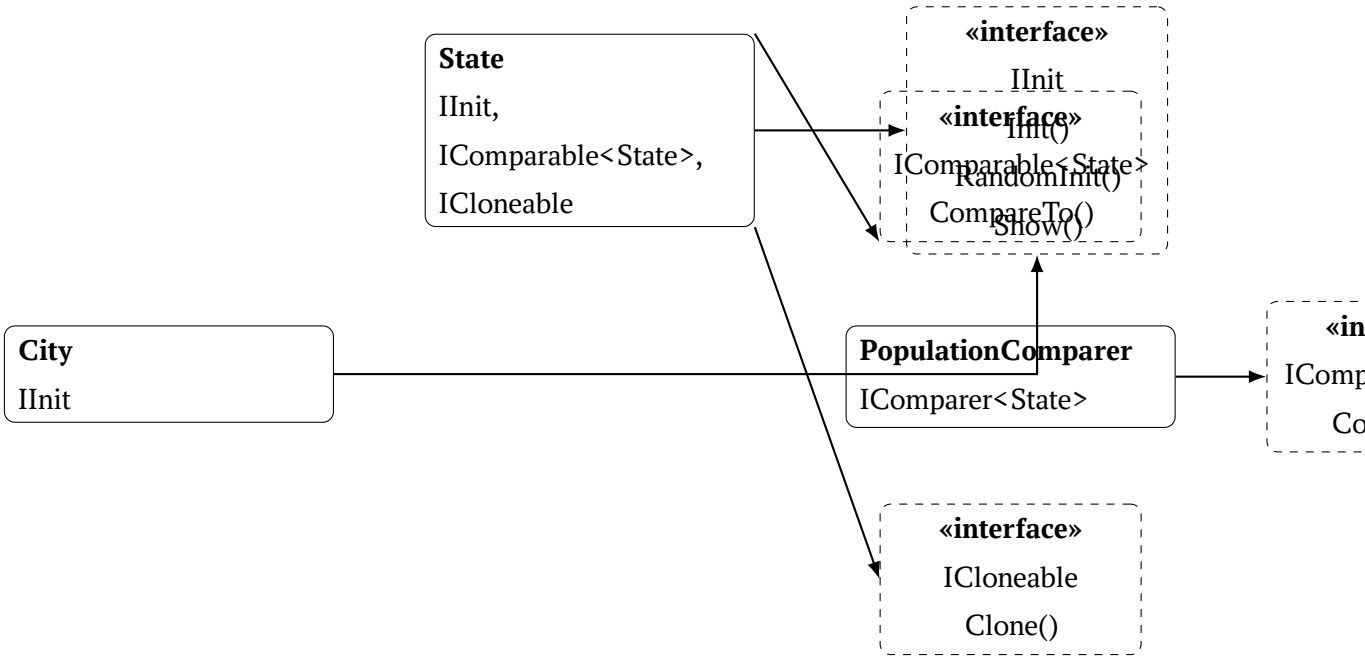


Рисунок 2.3 — Диаграмма интерфейсов и классов третьей части

2.4 Реализация первой части

В библиотеке создан базовый класс `State` и производные классы `Monarchy`, `Kingdom` и `Republic`. Свойства проверяют корректность значений: названия и имена не должны быть пустыми, численность населения и площадь не могут быть отрицательными, срок полномочий республики должен быть положительным.

В классах предусмотрены конструкторы без параметров, с параметрами и копирования, методы `Show()`, `Init()`, `RandomInit()` и переопределённый `Equals()`. Методы отображения, ввода и случайной инициализации объявлены виртуальными в базовом классе и переопределены в производных классах.

В демонстрационном приложении создаётся массив из 20 объектов типа `State`: по пять экземпляров каждого из четырёх типов. Для заполнения используется случайная инициализация; в первом задании каждый пятый объект дополнительно заполняется с клавиатуры. При обращении к виртуальному методу `Show()` через ссылку базового типа вызывается реализация, соответствующая фактическому типу объекта.

В исходном проекте отдельный невиртуальный метод для параллельного сравнения не реализован. Разница состоит в механизме выбора реализации: для виртуального метода она определяется фактическим типом объекта во время выполнения, а для невиртуального — типом ссылки/переменной на этапе компиляции.

2.5 Реализация второй части

Класс `Requests` содержит три статических метода. Метод `capitals()` проходит по всему массиву и формирует структуру `stateCapital`, содержащую название государства и название столицы. Метод `monarches()` проверяет каждый объект оператором `is`; если объект относится к типу `Monarchy` (включая экземпляры `Kingdom`), его название добавляется в результирующий список. Метод `republicHeadsTermSmallerN()` выбирает объекты типа `Republic`, приводит их к соответствующему типу и добавляет имя главы, если `TermLength < n`.

Параметр n вводится пользователем. Остальные объекты создаются автоматически с использованием генератора случайных чисел, что сокращает объём ручного ввода.

2.6 Реализация третьей части

Интерфейс `IInit` определяет методы `Init()`, `RandomInit()` и в фактической реализации также `Show()`. Класс `State` дополнительно реализует `IComparable<State>` и `ICloneable`. Метод `CompareTo()` сравнивает государства по площади. Поэтому вызов `Array.Sort(states)` сортирует массив по возрастанию площади.

Для сортировки по численности населения создан класс `PopulationComparer`. Его метод `Compare()` сравнивает свойства `Population`; сортировка выполняется вызовом `Array.Sort(states, new PopulationComparer())`.

После сортировки по площади выполняется бинарный поиск методом `Array.BinarySearch()`. Для корректной работы такого поиска массив дол-

жен быть предварительно отсортирован по тому же критерию, который использует `CompareTo()`.

Для демонстрации интерфейса создаётся массив `IInit`, содержащий экземпляры классов `State`, `Monarchy`, `Kingdom`, `Republic` и `City`. В цикле вызываются методы инициализации, после чего объекты отображаются через общий интерфейс.

В программе также вызываются `ShallowCopy()` и `Clone()`. Следует отметить особенность исходной реализации: метод `Clone()` в классе `State` копирует объект посредством `MemberwiseClone()` и сохраняет ту же ссылку на объект `Capital`. Поэтому фактически реализованное клонирование является поверхностным, а не глубоким. В результате изменение имени столицы через исходный объект отражается и в копии, полученной методом `Clone()`. Для глубокого копирования потребовалось бы создать отдельную копию объекта `City`.

3 Листинги исходного кода

В листингах 3.1-3.4 приведены файлы проекта. Исходные файлы включены в каталог `src` рядом с данным документом.

3.1 Библиотека классов

lst:class1

```
namespace lab_10_library
{

    public interface IInit
    {
        void Init();
        void RandomInit();
        void Show();

    }

    public class PopulationComparer : IComparer<State>
    {
        public int Compare(State? state1, State? state2)
        {

            return state1.Population.CompareTo(state2.Population);
        }
    }

    public class State : IInit, IComparable<State>, ICloneable
    {
        private string name;
        private City capital;
        private long population;
        private long area;

        public State ShallowCopy()
        {
            return (State)this.MemberwiseClone();
        }

        public virtual object Clone()
        {
            State copy = (State)this.MemberwiseClone();
            copy.capital = this.capital;
            return copy;
        }


        public string Name
        {
            get { return name; }
            set
            {
                if (value == null || value.Trim() == "")
                {
                    throw new ArgumentException("Название не может быть пустым");
```

```
            }
            name = value;
        }
    }

    public City Capital
    {
        get { return capital; }
        set
        {
            capital = value;
        }
    }
    public long Population
    {
        get { return population; }
        set
        {
            if (value < 0)
            {
                throw new ArgumentOutOfRangeException(nameof(value), "Население не может быть отрицательным");
            }
            population = value;
        }
    }
    public long Area
    {
        get { return area; }
        set
        {
            if (value < 0)
            {
                throw new ArgumentOutOfRangeException(nameof(value), "Площадь не может быть отрицательной");
            }
            area = value;
        }
    }

    public State()
    {
        Name = "Россия";
        Capital = new City("Москва");
        Population = 140000000;
        Area = 17125191;
    }

    public State(string name, string capital, long population, long area)
    {
        this.Name = name;
        this.Capital = new City(capital);
        this.Population = population;
        this.Area = area;
    }

    public State(State state)
    {
        Name = state.Name;
        Capital = state.Capital;
        Population = state.Population;
        Area = state.Area;
```

```
109         }
110
111         public virtual void Show()
112         {
113             Console.WriteLine($"Государство: {Name}");
114             Console.WriteLine($"Столица: {Capital.Name}");
115             Console.WriteLine($"Население: {Population}");
116             Console.WriteLine($"Площадь: {Area} км");
117         }
118         public virtual void Init()
119         {
120             Console.Write("Название: ");
121             Name = Console.ReadLine();
122             Console.Write("Столица: ");
123             Capital = new City(Console.ReadLine());
124             Console.Write("Население: ");
125             Population = readNotNegativeLong();
126             Console.Write("Площадь: ");
127             Area = readNotNegativeLong();
128         }
129
130         private static readonly Random rnd = new Random();
131         protected static Random Rnd { get { return rnd; } }
132
133         protected static string[] Names = { "Мондштадт", "Страна Пьяниц", "Дураковия", "Снежная", "Грустния" };
134         protected static string[] Capitals = { "Грязновск", "Сан-Фиерро", "Нищета", "Ельнино", "Свинск" };
135         public virtual void RandomInit()
136         {
137             Name = Names[Rnd.Next(Names.Length)];
138             Capital = new City(Capitals[rnd.Next(Capitals.Length)]);
139             Population = Rnd.Next(1000000, 100000000);
140             Area = Rnd.Next(1000000, 100000000);
141         }
142         public override bool Equals(object obj)
143         {
144             if (obj == null || !(obj is State))
145             {
146                 return false;
147             }
148             State other = (State)obj;
149             return Name == other.Name
150                 && Capital == other.Capital
151                 && Population == other.Population
152                 && Math.Abs(Area - other.Area) < 1e-5;
153         }
154
155         public override int GetHashCode()
156         {
157             return HashCode.Combine(Name, Capital, Population, Area);
158         }
159
160         public int CompareTo(State other)
161         {
162             if (other == null)
163             {
164                 return 1;
165             }
166             return Area.CompareTo(other.Area);
167         }
168
```

```
169         private static long readNotNegativeLong()
170         {
171             string input = Console.ReadLine();
172             long res;
173             bool parsed = long.TryParse(input, out res);
174             while (!parsed || res < 0)
175             {
176                 Console.WriteLine("Обшибка, введите повторно:");
177                 input = Console.ReadLine();
178                 parsed = long.TryParse(input, out res);
179             }
180             return res;
181         }
182     }
183
184
185     public class Monarchy : State
186     {
187         private string monarchName;
188         private bool constitutional;
189         public string MonarchName
190         {
191             get => monarchName;
192             set
193             {
194                 if (value == null || value.Trim() == "")
195                 {
196                     throw new ArgumentException("Имя монарха не может быть пустым");
197                 }
198                 monarchName = value;
199             }
200         }
201         public bool Constitutional
202         {
203             get => constitutional;
204             set => constitutional = value;
205         }
206
207         public Monarchy() : base()
208         {
209             MonarchName = "Монарх 1";
210             Constitutional = false;
211         }
212
213         public Monarchy(string name, string capital, long population, long area, string monarchName, bool
        ↪  constitutional)
214             : base(name, capital, population, area)
215         {
216             MonarchName = monarchName;
217             Constitutional = constitutional;
218         }
219
220         public Monarchy(Monarchy monarchy) : base(monarchy)
221         {
222             MonarchName = monarchy.MonarchName;
223             Constitutional = monarchy.Constitutional;
224         }
225
226         public override void Show()
227         {
```

```
228             base.Show();
229             Console.WriteLine($"Монарх: {MonarchName}");
230             Console.WriteLine($"Правление: {(Constitutional ? "конституционная" : "абсолютная")}");
231         }
232 
233         public override void Init()
234         {
235             base.Init();
236             Console.Write("Имя монарха: ");
237             MonarchName = Console.ReadLine();
238             Console.Write("Конституционная: ");
239             Constitutional = int.Parse(Console.ReadLine()) == 1;
240         }
241 
242         private static readonly string[] MonarchNames = { "Император Тупослав I", "Император Тупослав II", "Король
    ↪  Кощей V", "Царь Пьер Кровавый" };
243         public override void RandomInit()
244         {
245             base.RandomInit();
246             MonarchName = MonarchNames[Rnd.Next(MonarchNames.Length)];
247             Constitutional = Rnd.Next(2) == 1;
248         }
249 
250         public override bool Equals(object obj)
251         {
252             if (!base.Equals(obj)) { return false; }
253             Monarchy other = (Monarchy)obj;
254             return MonarchName == other.MonarchName && Constitutional == other.Constitutional;
255         }
256 
257         public override int GetHashCode()
258         {
259             return HashCode.Combine(base.GetHashCode(), MonarchName, Constitutional);
260         }
261     }
262 
263     public class Kingdom : Monarchy
264     {
265         private string dynasty;
266         public string Dynasty
267         {
268             get => dynasty;
269             set
270             {
271                 if (value == null || value.Trim() == "")
272                 {
273                     throw new ArgumentException("Династия не может быть пустым значением");
274                 }
275                 dynasty = value;
276             }
277         }
278         public Kingdom() : base()
279         {
280             Dynasty = "Рюриковичи";
281         }
282         public Kingdom(string name, string capital, long population, long area, string monarchName, bool
    ↪  constitutional, string dynasty)
283         : base(name, capital, population, area, monarchName, constitutional)
284         {
285             Dynasty = dynasty;
```

```
286         }
287         public Kingdom(Kingdom kingdom) : base(kingdom)
288         {
289             Dynasty = kingdom.Dynasty;
290         }
291         public override void Show()
292         {
293             base.Show();
294             Console.WriteLine($"Династия: {Dynasty}");
295         }
296         public override void Init()
297         {
298             base.Init();
299             Console.Write("Династия: ");
300             Dynasty = Console.ReadLine();
301         }
302 
303         private static readonly string[] dynasties =
304         { "Неглядовы", "Забываловы", "Яренских", "Николаевы" };
305 
306         public override void RandomInit()
307         {
308             base.RandomInit();
309             Dynasty = dynasties[Rnd.Next(dynasties.Length)];
310         }
311         public override bool Equals(object obj)
312         {
313             if (!base.Equals(obj)) { return false; }
314             Kingdom other = (Kingdom)obj;
315             return Dynasty == other.Dynasty;
316         }
317         public override int GetHashCode()
318         { return HashCode.Combine(base.GetHashCode(), Dynasty); }
319     }
320     public class Republic : State
321     {
322         private string headName;
323         private int termLength;
324         public string HeadName
325         {
326             get => headName;
327             set
328             {
329                 if (value == null || value.Trim() == "")
330                 {
331                     throw new ArgumentException("Имя главы не может быть пустым значением");
332                 }
333                 headName = value;
334             }
335         }
336         public int TermLength
337         {
338             get => termLength;
339             set
340             {
341                 if (value <= 0)
342                 {
343                     throw new ArgumentOutOfRangeException(nameof(value), "Срок полномочий должен быть больше 0");
344                 }
345                 termLength = value;
```

```
            }
        }
        public Republic() : base()
        {
            HeadName = "Путин";
            TermLength = 6;
        }

        public Republic(string name, string capital, long population, long area,
                    string headTitle, string headName, int termLength)
        : base(name, capital, population, area)
        {
            HeadName = headName;
            TermLength = termLength;
        }
        public Republic(Republic rep) : base(rep)
        {
            HeadName = rep.HeadName;
            TermLength = rep.TermLength;
        }

        public override void Show()
        {
            base.Show();
            Console.WriteLine($"Глава: {HeadName}");
            Console.WriteLine($"Срок: {TermLength} лет");
        }

        public override void Init()
        {
            base.Init();
            Console.Write("Имя главы: ");
            HeadName = Console.ReadLine();
            Console.Write("Срок полномочий: ");
            TermLength = readNotNegativeInt();
        }

        private static readonly string[] HeadNames = { "Застоев", "Движухов", "Мигрантов", "Коммунистов" };

        public override void RandomInit()
        {
            base.RandomInit();
            HeadName = HeadNames[Rnd.Next(HeadNames.Length)];
            TermLength = Rnd.Next(1, 8);
        }

        public override bool Equals(object obj)
        {
            if (!base.Equals(obj)) return false;
            Republic other = (Republic)obj;
            return HeadName == other.HeadName && TermLength == other.TermLength;
        }

        public override int GetHashCode()
        { return HashCode.Combine(base.GetHashCode(), HeadName, TermLength); }

        private static int readNotNegativeInt()
        {
            string input = Console.ReadLine();
            int res;
```

```
406             bool parsed = int.TryParse(input, out res);
407             while (!parsed || res < 0)
408             {
409                 Console.WriteLine("Обшибка, введите повторно:");
410                 input = Console.ReadLine();
411                 parsed = int.TryParse(input, out res);
412             }
413             return res;
414         }
415     }
416
417     public class City : IInit
418     {
419         private string name;
420         private long population;
421         private long area;
422         public string Name
423         {
424             get { return name; }
425             set
426             {
427                 if (value == null || value.Trim() == "")
428                 {
429                     throw new ArgumentException("Название города не может быть пустым");
430                 }
431                 name = value;
432             }
433         }
434         public long Population
435         {
436             get { return population; }
437             set
438             {
439                 if (value < 0)
440                 {
441                     throw new ArgumentException("Население не может быть отрицательным");
442                 }
443                 population = value;
444             }
445         }
446         public long Area
447         {
448             get { return area; }
449             set
450             {
451                 if (value < 0)
452                     throw new ArgumentException("Площадь не может быть отрицательной");
453                 area = value;
454             }
455         }
456
457         public City()
458         {
459             Name = "Город";
460             Population = 0;
461             Area = 0;
462         }
463         public City(string name)
464         {
465             this.Name = name;
```

```
            Population = 0;
            Area = 0;
        }
        public virtual void Init()
        {
            Console.Write("Название: ");
            Name = Console.ReadLine();
            Console.Write("Население: ");
            Population = readNotNegativeLong();
            Console.Write("Площадь: ");
            Area = readNotNegativeLong();
        }

        private static readonly Random rnd = new Random();

        protected static string[] Names = { "Залеск", "Дуркостан", "Пермь", "Тейково", "Город" };
        public virtual void RandomInit()
        {
            Name = Names[rnd.Next(Names.Length)];
            Population = rnd.Next(1000000, 100000000);
            Area = rnd.Next(1000000, 100000000);
        }

        public virtual void Show()
        {
            Console.WriteLine($"Город: {Name}");
            Console.WriteLine($"Население: {Population}");
            Console.WriteLine($"Площадь: {Area} км");
        }

        private static long readNotNegativeLong()
        {
            string input = Console.ReadLine();
            long res;
            bool parsed = long.TryParse(input, out res);
            while (!parsed || res < 0)
            {
                Console.WriteLine("Обшибка, введите повторно:");
                input = Console.ReadLine();
                parsed = long.TryParse(input, out res);
            }
            return res;
        }

    }
}
```

3.2 Основная программа

lst:program

```
using lab_10;
using lab_10_library;
using System.Net.Http.Headers;

int select;
bool parsed;

State[] states = new State[20];
```

```
for (var i = 0; i < 5; i++)
{
    states[i] = new State();
}
for (var i = 5; i < 10; i++)
{
    states[i] = new Monarchy();
}
for (var i = 10; i < 15; i++)
{
    states[i] = new Kingdom();
}
for (var i = 15; i < 20; i++)
{
    states[i] = new Republic();
}

for (var i = 0; i < states.Length; i++)
{
    states[i].RandomInit();
}
do
{
    parsed = UserInterface.selectMenu(out select);
    if (!parsed)
    {
        break;
    }

    switch (select)
    {
        case 1:
            {
                Console.Clear();
                task1();
                UserInterface.wait();
                break;
            }
        case 2:
            {
                Console.Clear();
                task2();
                UserInterface.wait();
                break;
            }
        case 3:
            Console.Clear();
            task3();
            UserInterface.wait();
            break;
    }
} while (select >= 1 && select <= 3);



void task1()
{


    for (var i = 0; i < 20; i++)
```

```
    {
        selectInit(ref states[i], i);
    }

    for (var i = 0; i < 20; i++)
    {
        states[i].Show();
        Console.WriteLine();
    }


}

void selectInit(ref State state, int i)
{
    if (i % 5 == 0)
    {
        states[i].Init();
    }
    else
    {
        states[i].RandomInit();
    }
}

void task2()
{
    int request = UserInterface.selectRequest();
    switch (request)
    {
        case 1:
            Console.Clear();
            Requests.stateCapital[] capitals;
            Requests.capitals(out capitals, ref states);
            for (var i = 0; i < capitals.Length; i++)
            {
                Console.WriteLine($"Столица страны {capitals[i].StateName} : {capitals[i].Capital}");
            }
            UserInterface.wait();
            break;
        case 2:
            Console.Clear();
            List<string> monarches;
            Requests.monarches(out monarches, ref states);
            Console.Write("Монархии: ");
            for (var i = 0; i < monarches.Count; i++)
            {
                Console.Write($"{monarches[i]}, ");
            }
            Console.WriteLine();
            break;
        case 3:
            Console.Clear();
            Console.Write("Введите срок n для поиска: ");
            int n = UserInterface.readInt((res) => (res > 0));
            List<string> heads;
            Requests.republicHeadsTermSmallerN(out heads, n, ref states);
            Console.WriteLine($"Лидеры республик со сроком меньше {n}: ");
            for (var i = 0; i < heads.Count; i++)
            {
```

```
                Console.Write($"{heads[i]} ");
            }
            Console.WriteLine();
            break;
    }
}

void task3()
{


    Array.Sort(states, new PopulationComparer());
    Console.WriteLine("Массив, отсортироанный по численности населения стран: ");
    for (var i = 0; i < 20; i++)
    {
        states[i].Show();
        Console.WriteLine();
    }

    UserInterface.waitContinue();

    Array.Sort(states);
    Console.WriteLine("Массив, отсортироанный по площади стран: ");
    for (var i = 0; i < 20; i++)
    {
        states[i].Show();
        Console.WriteLine();
    }

    UserInterface.waitContinue();

    State targetState = new State();
    Console.WriteLine("Введите госуларство для поиска по площади: ");
    targetState.Init();
    int index = Array.BinarySearch(states, targetState);
    if (index >= 0)
    {
        Console.WriteLine($"Государство найдено под индексом {index}");
        states[index].Show();
    } else
    {
        Console.WriteLine("Государство не найдено");
    }

    UserInterface.waitContinue();

    Console.WriteLine("Объекты IInit:");

    IInit[] initArray = new IInit[10] {
        new State(),  new State (),
        new Monarchy(), new Monarchy(),
        new Kingdom(), new Kingdom(),
        new Republic(), new Republic(),
        new City(), new City()};
    for (var i = 5; i < 10; i++)
    {
        if (i % 2 == 0)
        {
            initArray[i].Init();
        } else
```

```
        {
            initArray[i].RandomInit();
        }
    }

    for (var i = 0; i < 10; i++)
    {
        initArray[i].Show();
        Console.WriteLine();
    }

    UserInterface.waitContinue();

    Console.WriteLine($"Создание клона и проверхностной копии первого госуарства");
    State original = states[0];
    Console.WriteLine("оригинал (states[0])");
    original.Show();
    Console.WriteLine();

    State shallow = original.ShallowCopy();
    State deep = (State)original.Clone();

    Console.WriteLine("Меняем столицу у оригинала на Санкт-Петербург");
    original.Capital.Name = "Санкт-Петербург";
    Console.WriteLine($"стлица оригинала: {original.Capital.Name}");
    Console.WriteLine($"столица поверхностной копии: {shallow.Capital.Name}");
    Console.WriteLine($"столица глубокой копии: {deep.Capital.Name}");
}
```

3.3 Класс запросов

lst:requests

```
using System;
using System.Collections.Generic;
using System.Text;

using lab_10_library;

namespace lab_10
{
    internal class Requests
    {
        public struct stateCapital
        {
            public string StateName;
            public string Capital;
        }
        public static void capitals(out stateCapital[] res, ref State[] states)
        {
            res = new stateCapital[states.Length];
            for (var i = 0; i < states.Length; i++)
            {
                res[i] = new() { StateName = states[i].Name, Capital = states[i].Capital.Name };
            }
        }

        public static void monarches(out List<string> res, ref State[] states)
        {
```

```
            res = new List<string>();
            for (var i = 0; i < states.Length; i++)
            {
                if (states[i] is Monarchy)
                {
                    res.Add(states[i].Name);
                }
            }
        }

        public static void republicHeadsTermSmallerN(out List<string> res, int n, ref State[] states)
        {
            res = new List<string>();
            for (var i = 0; i < states.Length; i++)
            {
                if (states[i] is Republic)
                {
                    Republic rep = (Republic)states[i];
                    if (rep.TermLength < n)
                    {
                        res.Add(rep.HeadName);
                    }
                }
            }
        }
    }
}
```

3.4 Пользовательский интерфейс

lst:ui

```
using System;

namespace lab_10
{
    internal static class UserInterface
    {

        public static bool selectMenu(out int inputInt)
        {
            Console.Clear();
            Console.WriteLine("1. Задача 1 (заполнение массива всеми объектами иерархии и вывод)");
            Console.WriteLine("2. Задача 2 (Выполнение запросов)");
            Console.WriteLine("3. Задача 3 (Сортировка, поиск, создание объектов интерфейса)");
            Console.Write("Выберите пункт 1-3 или введите любое другое значение для выхода: ");
            string? input = Console.ReadLine();
            return int.TryParse(input, out inputInt);
        }

        public static int selectRequest()
        {
            Console.Clear();
            Console.WriteLine("1. Столицы всех государств");
            Console.WriteLine("2. Все монархи");
            Console.WriteLine("3. Лидеры республик со сроком полномичй менее n");
            Console.Write("Выберите пункт 1-3: ");
            return readInt((res) => (res >= 1 && res <= 3));
        }
```

```
28
29
30          public static int readInt(Func<int, bool> condition)
31          {
32              while (true)
33              {
34                  string? input = Console.ReadLine();
35                  if (int.TryParse(input, out int res) && condition(res))
36                  {
37                      return res;
38                  }
39                  Console.Write("Ошибка! Введите заново: ");
40              }
41          }
42
43          public static void wait()
44          {
45              Console.WriteLine("Для выхода введите любое значение: ");
46              Console.ReadLine();
47          }
48          public static void waitContinue()
49          {
50              Console.WriteLine("Введите любое значение, чтобы продолжить: ");
51              Console.ReadLine();
52              Console.Clear() ;
53          }
54      }
55  }
```

Заключение

В ходе лабораторной работы разработана библиотека классов для представления государств, монархий, королевств и республик. Реализованы свойства с проверкой значений, конструкторы, методы инициализации, отображения и сравнения объектов. На основе иерархии создано демонстрационное приложение, использующее массив объектов базового типа.

Реализованы три запроса: получение столиц государств, отбор монархий и поиск глав республик с заданным ограничением срока полномочий. Дополнительно реализованы интерфейсы и механизмы сортировки по площади и численности населения, а также бинарный поиск.

В работе рассмотрено применение интерфейса `IInit` и класса `City`, не входящего в основную иерархию. Проверка исходного кода показывает, что метод `Clone()` в текущем виде не создаёт независимую копию объекта столицы; для демонстрации именно глубокого клонирования эту часть реализации следует дополнить копированием объекта `City`.